DESIGN AND ANALYSIS OF ALGORITHM

LAB EXPERIMENT - 6

NAME: INBARASU

ROLL NO:

CH.SC.U4CSE24242

1.AVL TREE

CODE:

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int key;
    struct Node *left;
    struct Node *right;
    int height;
};
int height(struct Node *N) {
    if (N == NULL)
        return 0;
    return N->height;
}
int max(int a, int b) {
    return (a > b) ? a : b;
}
struct Node* newNode(int key) {
    struct Node* node = (struct Node*)
        malloc(sizeof(struct Node));
    node->key = key;
    node->left = NULL;
    node->right = NULL;
    node->height = 1;
    return node;
}
struct Node *rightRotate(struct Node *y) {
    struct Node *x = y->left;
    struct Node *T2 = x->right;

    x->right = y;
    y->left = T2;

    y->height = max(height(y->left), height(y->right)) + 1;
    x->height = max(height(x->left), height(x->right)) + 1;

    return x;
}
struct Node *leftRotate(struct Node *x) {
    struct Node *y = x->right;
    struct Node *T2 = y->left;

    y->left = x;
    x->right = T2;

    x->height = max(height(x->left), height(x->right)) + 1;
    y->height = max(height(y->left), height(y->right)) + 1;

    return y;
}
int getBalance(struct Node *N) {
    if (N == NULL)
        return 0;
    return height(N->left) - height(N->right);
}
struct Node* insert(struct Node* node, int key) {

    if (node == NULL)
        return newNode(key);

    if (key < node->key)
```

```
    if (key < node->key)
        node->left = insert(node->left, key);
    else if (key > node->key)
        node->right = insert(node->right, key);
    else
        return node;

    node->height = 1 + max(height(node->left),
                           height(node->right));

    int balance = getBalance(node);
    if (balance > 1 && key < node->left->key)
        return rightRotate(node);
    if (balance < -1 && key > node->right->key)
        return leftRotate(node);
    if (balance > 1 && key > node->left->key) {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }
    if (balance < -1 && key < node->right->key) {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }

    return node;
}

void preorder(struct Node *root) {
    if(root != NULL) {
        printf("%d ", root->key);
        preorder(root->left);
        preorder(root->right);
    }
}
int main() {

    struct Node *root = NULL;
    root = insert(root, 10);
    root = insert(root, 20);
    root = insert(root, 30);
    root = insert(root, 40);
    root = insert(root, 50);
    root = insert(root, 25);
    printf("Roll no : ch.sc.u4cse24215\n");
    printf("Name : Hemal\n");
    printf("Preorder traversal of AVL tree:\n");
    preorder(root);
    return 0;
}
```

OUTPUT:

```
hemal@Admin:~$ nano avl.c
hemal@Admin:~$ gcc avl.c -o avl
hemal@Admin:~$ ./avl
Roll no : ch.sc.u4cse24215
Name : Hemal
Preorder traversal of AVL tree:
30 20 10 25 40 50 hemal@Admin:~$
```

INTERPRETATION:

An avl tree is a self-balancing binary search Tree introduced by Georgy Adelson-Velsky and Evgenii Landis where the height difference between left and right subtrees of every node is at most 1.
If the balance factor becomes greater than 1 or less than −1, rotations are performed to maintain the tree balance.

2. RED BLACK TREE

CODE:

```c
#include <stdio.h>
#include <stdlib.h>
struct node {
    int data;
    char color; // 'R' or 'B'
    struct node *left, *right, *parent;
};
struct node* root = NULL;
struct node* createNode(int data) {
    struct node* newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = data;
    newnode->color = 'R';
    newnode->left = newnode->right = newnode->parent = NULL;
    return newnode;
}
void leftRotate(struct node* x) {
    struct node* y = x->right;
    x->right = y->left;

    if (y->left != NULL)
        y->left->parent = x;

    y->parent = x->parent;

    if (x->parent == NULL)
        root = y;
    else if (x == x->parent->left)
        x->parent->left = y;
    else
        x->parent->right = y;

    y->left = x;
    x->parent = y;
}
void rightRotate(struct node* y) {
    struct node* x = y->left;
    y->left = x->right;

    if (x->right != NULL)
        x->right->parent = y;

    x->parent = y->parent;

    if (y->parent == NULL)
        root = x;
    else if (y == y->parent->left)
        y->parent->left = x;
    else
        y->parent->right = x;

    x->right = y;
    y->parent = x;
```

```
void fixInsert(struct node* z) {
    while (z->parent != NULL && z->parent->color == 'R') {

        struct node* grandparent = z->parent->parent;

        if (z->parent == grandparent->left) {

            struct node* uncle = grandparent->right;

            if (uncle != NULL && uncle->color == 'R') {
                z->parent->color = 'B';
                uncle->color = 'B';
                grandparent->color = 'R';
                z = grandparent;
            }
            else {
                if (z == z->parent->right) {
                    z = z->parent;
                    leftRotate(z);
                }

                z->parent->color = 'B';
                grandparent->color = 'R';
                rightRotate(grandparent);
            }
        }
        else {

            struct node* uncle = grandparent->left;

            if (uncle != NULL && uncle->color == 'R') {
                z->parent->color = 'B';
                uncle->color = 'B';
                grandparent->color = 'R';
                z = grandparent;
            }
            else {
                if (z == z->parent->left) {
                    z = z->parent;
                    rightRotate(z);
                }

                z->parent->color = 'B';
                grandparent->color = 'R';
                leftRotate(grandparent);
            }
        }
    }

    root->color = 'B';
}
void insert(int data) {
```

```c
void insert(int data) {

    struct node* z = createNode(data);
    struct node* y = NULL;
    struct node* x = root;

    while (x != NULL) {
        y = x;
        if (z->data < x->data)
            x = x->left;
        else
            x = x->right;
    }

    z->parent = y;

    if (y == NULL)
        root = z;
    else if (z->data < y->data)
        y->left = z;
    else
        y->right = z;

    fixInsert(z);
}
void inorder(struct node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d(%c) ", root->data, root->color);
        inorder(root->right);
    }
}
int main() {

    printf("Roll no : ch.sc.u4cse24215\n");
    printf("Name : Hemal\n");
    insert(10);
    insert(20);
    insert(30);
    insert(15);
    insert(25);
    printf("Inorder traversal of Red-Black Tree:\n");
    inorder(root);

    return 0;
}
```

OUTPUT:

```
hemal@Admin:~$ nano redblack.c
hemal@Admin:~$ gcc redblack.c -o redblack
hemal@Admin:~$ ./redblack
Roll no : ch.sc.u4cse24215
Name : Hemal
Inorder traversal of Red-Black Tree:
10(B) 15(R) 20(B) 25(R) 30(B) hemal@Admin:~$
```

INTERPRETATION:

A red black tree is a self-balancing binary search tree in which each node is colored either red or black to maintain balance. It follows rules such as no two consecutive red nodes and the same number of black nodes on every path from root to leaf.